Regression and Prediction of Concrete Strength

VIBE CIVIL 2000 · AI/ML for Sustainable Infrastructure Systems · Problem-1 · Q1.1

Name: Ranvir Kumar Roll No.: 23CE36016 IIT Kharagpur

1. Problem Statement

The compressive strength of concrete generally increases with curing age. For a particular mix, three 150 × 150 × 150 mm cubes were tested at each curing age; the failure loads are:

Curing age (days)	Cube 1 (kN)	Cube 2 (kN)	Cube 3 (kN)
3	385	395	400
7	525	535	545
14	655	665	680
21	740	755	760
28	800	815	830

Using AI-assisted Python programming, investigate the relationship between curing age and compressive strength through **Linear, Quadratic, Logarithmic and Exponential** regression; develop and compare the four models; predict the strength at **t = 10 days** and **t = 35 days**; and present the analysis, results, graphs and engineering interpretation. The Python program is to be submitted with the report.

2. Understanding (Workflow: Understand)

- **Unit conversion.** Strength f = P / A. A = 150 × 150 = 22,500 mm². With P in kN: f (N/mm² = MPa) = P × 1000 / 22,500. Example: 385 kN → 17.11 MPa.
- **Data.** 5 ages × 3 cubes = 15 observations (t, f). All 15 points are used in the regression (not just the means), so the fit also reflects cube-to-cube scatter.
- **Models.** Linear f = a + bt; Quadratic f = a + bt + ct²; Logarithmic f = a + b ln t; Exponential f = $A \cdot e^{kt}$ (fitted by least squares on ln f vs t).
- **Comparison criteria.** R^2, adjusted R^2 (penalises the extra parameter of the quadratic), RMSE, standard error, residual plots, and leave-one-age-out cross-validation RMSE (all 3 cubes of one age are held out, the model is refitted, and the held-out cubes are predicted). Physical plausibility is also checked.
- **Expected physics.** Strength gain is fast early (hydration) and flattens with time, so a concave, saturating curve is expected. The 28-day strength is the code reference, so t = 35 d is an extrapolation beyond the tested range (3–28 d).

3. Prompt(s) Used (Workflow: Prompt)

Prompt 1 (main prompt):

"*Write a complete Python program for the following civil-engineering problem. Data: curing ages 3, 7, 14, 21, 28 days; failure loads (kN) of three 150 mm cubes at each age: 3d: 385, 395, 400; 7d: 525, 535, 545; 14d: 655, 665, 680; 21d: 740, 755, 760; 28d: 800, 815, 830. (i) Convert load to compressive strength in MPa using the cube area 150×150 mm². (ii) Fit Linear, Quadratic, Logarithmic and Exponential regression models to all 15 points vs curing age using numpy/scipy. (iii) For each model report the equation, R², adjusted R², RMSE and a cross-validation error. (iv) Predict strength at t = 10 and 35 days. (v) Plot the data with all four fitted curves, a residual plot, and a prediction bar chart, with labelled axes and units. (vi) Add validation checks: unit check, compare numpy fit with the normal equations, compare linearised vs nonlinear exponential fit. Do not hide assumptions.*"

Prompt 2 (validation / interpretation):

"*Compare the four models. Which is physically most reasonable for concrete strength gain, and are the t = 10 d and t = 35 d predictions realistic? Check magnitudes against the typical 3-day and 7-day ratios to 28-day strength.*"

Note: the AI output was not accepted blindly; each result was re-checked as described in Section 6.

4. Final Code (Workflow: Compute)

File: **regression_analysis.py** (Python 3, numpy, scipy, matplotlib).

```
"""
Q1.1  Regression and Prediction of Concrete Strength
Curing age (days) vs compressive strength of 150 mm cubes.
Models: Linear, Quadratic, Logarithmic, Exponential.
Run:  python regression_analysis.py
Outputs: figures (PNG) + results.json in the same folder as this script.
"""
import json, os
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

OUT = os.path.dirname(os.path.abspath(__file__))

# ---------------- 1. DATA ----------------
age = np.array([3, 7, 14, 21, 28], dtype=float)                    # days
load = np.array([[385, 395, 400],
                 [525, 535, 545],
                 [655, 665, 680],
                 [740, 755, 760],
                 [800, 815, 830]], dtype=float)                     # kN
AREA = 150.0 * 150.0                                               # mm^2 (cube face)
stress = load * 1e3 / AREA                                         # N/mm^2 = MPa
t = np.repeat(age, 3)                                              # 15 observations
f = stress.flatten()
n = len(f)
T_PRED = np.array([10.0, 35.0])

# ---------------- 2. MODELS ----------------
def r2(y, yh):  return 1 - np.sum((y - yh) ** 2) / np.sum((y - y.mean()) ** 2)
def adj(r, n, p): return 1 - (1 - r) * (n - 1) / (n - p - 1)       # p = predictors

models = {}

# Linear: f = a + b t
b1, a1 = np.polyfit(t, f, 1)
models["Linear"] = dict(fn=lambda x, a=a1, b=b1: a + b * x, p=1,
                        eq=f"f = {a1:.3f} + {b1:.4f} t", coef=dict(a=a1, b=b1))

# Quadratic: f = a + b t + c t^2
c2, b2, a2 = np.polyfit(t, f, 2)
models["Quadratic"] = dict(fn=lambda x, a=a2, b=b2, c=c2: a + b * x + c * x ** 2, p=2,
                           eq=f"f = {a2:.3f} + {b2:.4f} t {c2:+.5f} t^2", coef=dict(a=a2, b=b2, c=c2))

# Logarithmic: f = a + b ln t
b3, a3 = np.polyfit(np.log(t), f, 1)
models["Logarithmic"] = dict(fn=lambda x, a=a3, b=b3: a + b * np.log(x), p=1,
                             eq=f"f = {a3:.3f} + {b3:.4f} ln(t)", coef=dict(a=a3, b=b3))

# Exponential: f = A e^(k t)  -> ln f = ln A + k t (linearised least squares)
k4, lnA4 = np.polyfit(t, np.log(f), 1)
A4 = np.exp(lnA4)
models["Exponential"] = dict(fn=lambda x, A=A4, k=k4: A * np.exp(k * x), p=1,
                             eq=f"f = {A4:.3f} exp({k4:.5f} t)", coef=dict(A=A4, k=k4))

# ---------------- 3. METRICS ----------------
results = {}
for name, m in models.items():
    yh = m["fn"](t)
    res = f - yh
    R2 = r2(f, yh)
    sse = float(np.sum(res ** 2))
    dof = n - (m["p"] + 1)
    # leave-one-age-out CV (drop all 3 cubes of one age, refit, predict them)
    cv_err = []
    for ag in age:
        tr = t != ag
        tt, ff = t[tr], f[tr]
        if name == "Linear":       c = np.polyfit(tt, ff, 1);           pr = np.polyval(c, ag)
        elif name == "Quadratic":  c = np.polyfit(tt, ff, 2);           pr = np.polyval(c, ag)
        elif name == "Logarithmic":c = np.polyfit(np.log(tt), ff, 1);   pr = np.polyval(c, np.log(ag))
        else:                      c = np.polyfit(tt, np.log(ff), 1);   pr = np.exp(np.polyval(c, ag))
        cv_err.extend(f[t == ag] - pr)
    results[name] = dict(
        equation=m["eq"], coef={k: float(v) for k, v in m["coef"].items()},
        R2=float(R2), adjR2=float(adj(R2, n, m["p"])),
        SSE=sse, RMSE=float(np.sqrt(sse / n)), SE=float(np.sqrt(sse / dof)),
        MAE=float(np.mean(np.abs(res))), CV_RMSE=float(np.sqrt(np.mean(np.square(cv_err)))),
        pred10=float(m["fn"](10.0)), pred35=float(m["fn"](35.0)),
        pred28=float(m["fn"](28.0)),
        residuals=[float(x) for x in res])

# ---------------- 4. VALIDATION CHECKS ----------------
checks = {}
# (a) polyfit vs explicit normal equations for the linear model
X = np.c_[np.ones(n), t]
beta = np.linalg.solve(X.T @ X, X.T @ f)
checks["linear_normal_eq_vs_polyfit"] = [float(beta[0]), float(a1), float(beta[1]), float(b1)]
# (b) true nonlinear LSQ for exponential, compared with linearised version
popt, _ = curve_fit(lambda x, A, k: A * np.exp(k * x), t, f, p0=[A4, k4])
yh_nl = popt[0] * np.exp(popt[1] * t)
checks["exp_nonlinear"] = dict(A=float(popt[0]), k=float(popt[1]), R2=float(r2(f, yh_nl)),
                               pred10=float(popt[0] * np.exp(popt[1] * 10)),
                               pred35=float(popt[0] * np.exp(popt[1] * 35)))
```

```
# (c) hand checks on the raw data
mean_strength = stress.mean(axis=1)
checks["mean_strength_by_age"] = [float(x) for x in mean_strength]
checks["ratio_to_28d"] = [float(x) for x in mean_strength / mean_strength[-1]]
checks["unit_check_385kN"] = float(385e3 / AREA)
checks["quad_vertex_t"] = float(-b2 / (2 * c2)) if c2 != 0 else None
# (d) supplementary saturating model (not requested; used only for discussion)
sat, _ = curve_fit(lambda x, fi, k: fi * (1 - np.exp(-k * x)), t, f, p0=[45, 0.1])
yh_sat = sat[0] * (1 - np.exp(-sat[1] * t))
checks["saturating_extra"] = dict(finf=float(sat[0]), k=float(sat[1]), R2=float(r2(f, yh_sat)),
                                  pred10=float(sat[0] * (1 - np.exp(-sat[1] * 10))),
                                  pred35=float(sat[0] * (1 - np.exp(-sat[1] * 35))))

json.dump(dict(results=results, checks=checks,
               data=dict(age=age.tolist(), load=load.tolist(), stress=stress.tolist())),
          open(os.path.join(OUT, "results.json"), "w"), indent=2)

# ---------------- 5. PLOTS ----------------
colors = {"Linear": "tab:blue", "Quadratic": "tab:green", "Logarithmic": "tab:red", "Exponential": "tab:purple"}
tt = np.linspace(1, 40, 400)

fig, ax = plt.subplots(figsize=(8, 5.2))
ax.scatter(t, f, c="k", s=28, zorder=5, label="Experimental (15 cubes)")
ax.scatter(age, mean_strength, marker="D", facecolor="none", edgecolor="k", s=70, zorder=5, label="Mean at each age")
for nm, m in models.items():
    ax.plot(tt, m["fn"](tt), color=colors[nm], lw=1.8, label=f"{nm} (R² = {results[nm]['R2']:.4f})")
ax.axvspan(28, 40, color="0.92", zorder=0); ax.text(28.6, 3, "extrapolation\nbeyond data", fontsize=8, color="0.4")
for tp, ls in zip(T_PRED, [":", ":"]): ax.axvline(tp, color="0.5", ls=ls, lw=0.9)
ax.set_xlabel("Curing age, t (days)"); ax.set_ylabel("Compressive strength, f (N/mm² = MPa)")
ax.set_title("Compressive strength vs curing age: four regression models")
ax.set_xlim(0, 40); ax.set_ylim(0, None); ax.grid(alpha=0.3); ax.legend(fontsize=8, loc="upper left")
fig.tight_layout(); fig.savefig(os.path.join(OUT, "fig1_fits.png"), dpi=200); plt.close(fig)

fig, axs = plt.subplots(2, 2, figsize=(8, 5.6), sharex=True, sharey=True)
for ax, (nm, m) in zip(axs.flat, models.items()):
    ax.axhline(0, color="k", lw=0.8); ax.scatter(t, results[nm]["residuals"], color=colors[nm], s=26)
    ax.set_title(f"{nm}  (RMSE = {results[nm]['RMSE']:.2f} MPa)", fontsize=9); ax.grid(alpha=0.3)
for ax in axs[1]: ax.set_xlabel("Curing age, t (days)")
for ax in axs[:, 0]: ax.set_ylabel("Residual, f − f_fit (MPa)")
fig.suptitle("Residuals of the four models", fontsize=11); fig.tight_layout()
fig.savefig(os.path.join(OUT, "fig2_residuals.png"), dpi=200); plt.close(fig)

fig, ax = plt.subplots(figsize=(8, 4.6))
names = list(models); w = 0.38; x = np.arange(len(names))
b_a = ax.bar(x - w / 2, [results[n_]["pred10"] for n_ in names], w, label="t = 10 d", color="tab:cyan")
b_b = ax.bar(x + w / 2, [results[n_]["pred35"] for n_ in names], w, label="t = 35 d", color="tab:orange")
for bars in (b_a, b_b):
    for r in bars: ax.text(r.get_x() + r.get_width() / 2, r.get_height() + 0.5, f"{r.get_height():.1f}", ha="center", fontsize=8)
ax.set_xticks(x); ax.set_xticklabels(names); ax.set_ylabel("Predicted strength (MPa)")
ax.set_title("Predictions at t = 10 d and t = 35 d"); ax.grid(axis="y", alpha=0.3); ax.legend()
fig.tight_layout(); fig.savefig(os.path.join(OUT, "fig3_predictions.png"), dpi=200); plt.close(fig)

# ---------------- 6. CONSOLE SUMMARY ----------------
print(f"Area = {AREA:.0f} mm^2; 385 kN -> {385e3/AREA:.3f} MPa")
print("Mean strength by age (MPa):", np.round(mean_strength, 2))
for nm, r in results.items():
    print(f"{nm:12s} {r['equation']:48s} R2={r['R2']:.4f} adjR2={r['adjR2']:.4f} "
          f"RMSE={r['RMSE']:.3f} CV_RMSE={r['CV_RMSE']:.3f} f(10)={r['pred10']:.2f} f(35)={r['pred35']:.2f}")
print("Checks:", json.dumps(checks, indent=1))
```

5. Output

5.1 Strength data (MPa) and mean strength

Age (d)	Cube 1	Cube 2	Cube 3	Mean	Mean / f28
3	17.11	17.56	17.78	17.48	0.48
7	23.33	23.78	24.22	23.78	0.66
14	29.11	29.56	30.22	29.63	0.82
21	32.89	33.56	33.78	33.41	0.92
28	35.56	36.22	36.89	36.22	1.00

5.2 Fitted models

Model	Equation (f in MPa, t in days)	R^2	Adj. R^2	RMSE (MPa)	CV-RMSE (MPa)
Linear	f = 17.577 + 0.7210 t	0.9389	0.9342	1.672	3.206
Quadratic	f = 13.862 + 1.4639 t -0.02411 t^2	0.9900	0.9884	0.675	1.998
Logarithmic	f = 7.904 + 8.3746 ln t	0.9935	0.9930	0.545	0.894
Exponential	f = 18.278 exp(0.02723 t)	0.8811	0.8720	2.332	5.015

5.3 Predictions

Model	f(10 d) MPa	f(35 d) MPa	f(28 d) fitted MPa	Observed mean f28
Linear	24.79	42.81	37.76	36.22
Quadratic	26.09	35.57	35.95	36.22
Logarithmic	27.19	37.68	35.81	36.22
Exponential	24.00	47.41	39.18	36.22

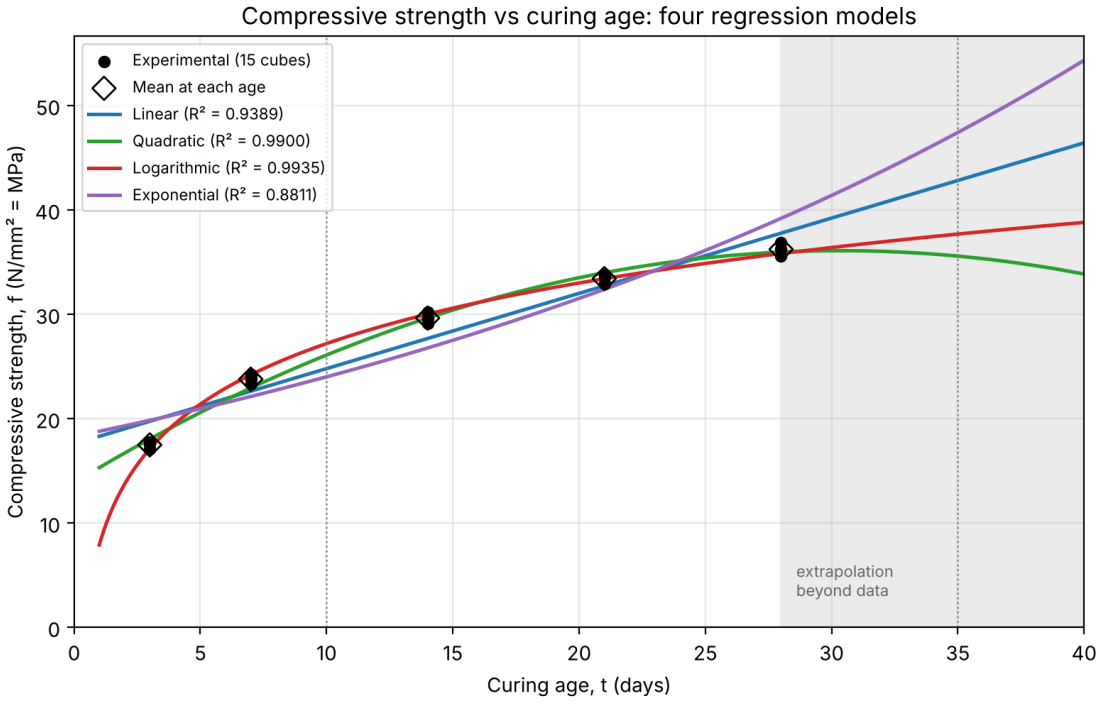


Fig. 1: Experimental data and the four regression curves (shaded region = extrapolation beyond tested ages).

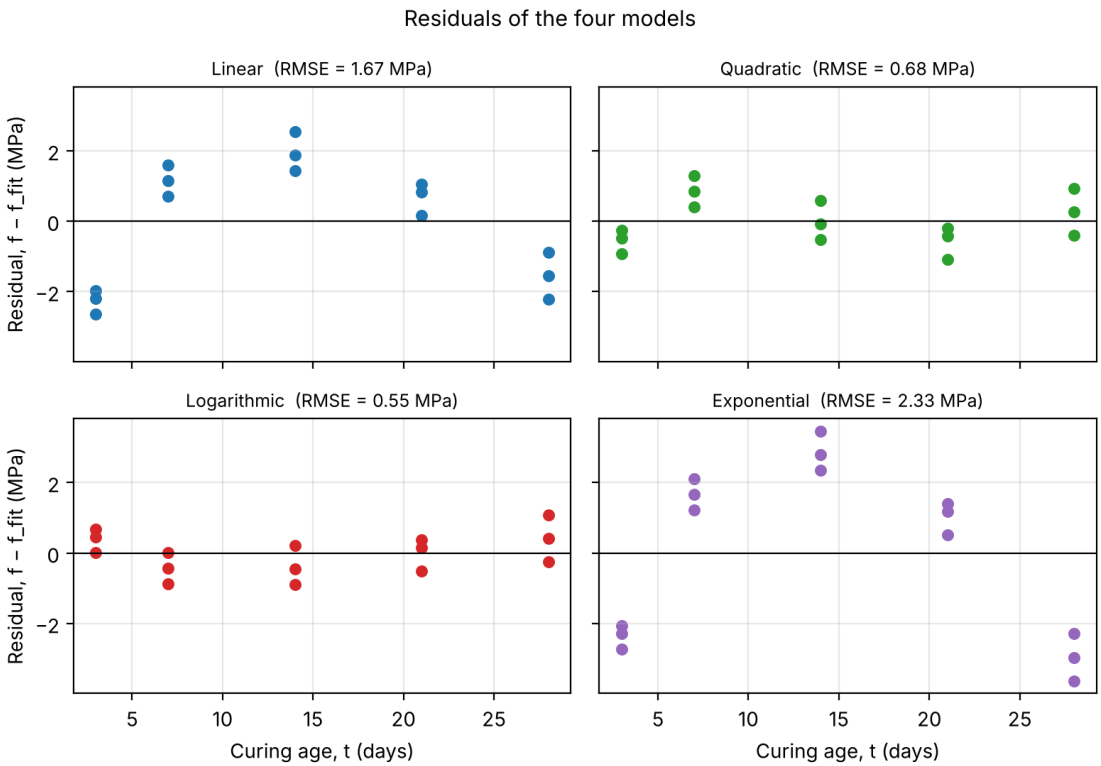


Fig. 2: Residuals of each model. Linear and exponential show systematic curved patterns; quadratic and logarithmic are small and mostly random.

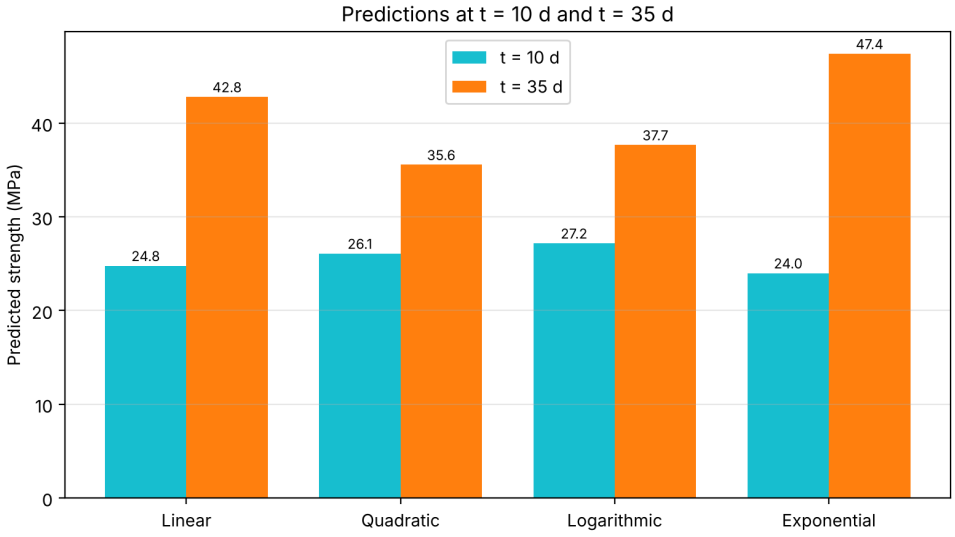


Fig. 3: Predicted strength at t = 10 d and 35 d.

6. Validation and Interpretation (Workflow: Validate)

6.1 Validation checks

- **Units/magnitude:** 385 kN / 22,500 mm² = 17.11 MPa (hand calc matches code). 28-day mean = 36.2 MPa, typical of an M30–M35 concrete. Strength at 3 d is 48% and at 7 d is 66% of the 28-day value, which is consistent with normal curing behaviour (roughly 40–50% at 3 d and 65–70% at 7 d).
- **Linear model:** the numpy fit was reproduced by solving the normal equations $(X^TX)\beta = X^Tf$: a = 17.5773, b = 0.72099 in both.
- **Exponential model:** the linearised fit (ln f vs t) minimises error in ln f, not in f. A true nonlinear least-squares fit gives $f = 19.32 \cdot \exp(0.02410\ t)$, $R^2 = 0.8959$, with predictions 24.59 MPa (10 d) and 44.91 MPa (35 d), versus 24.00 and 47.41 MPa from the linearised version. Both are poor, so the conclusion does not change.
- **Interpolation sanity:** all models give f(10 d) between 24 and 27.2 MPa, which lies between the observed 7-d mean (23.8) and 14-d mean (29.6), as it should.

6.2 Comparison of models

Model	Fit quality	Residual pattern	Physical realism
Linear	$R^2 = 0.939$	Systematic curve (+, −, +)	Strength grows without bound; overpredicts at 35 d (42.8 MPa, +18% over the 28-d value)
Quadratic	$R^2 = 0.990$	Small, mostly random	Peaks at t ≈ 30.4 d, then decreases; 35-d value (35.6) is below the 28-d value, which is unrealistic for moist-cured concrete
Logarithmic	$R^2 = 0.9935$ (best)	Smallest, random	Concave, decelerating gain; 35-d value 37.7 MPa (+4% over 28 d) is realistic
Exponential	$R^2 = 0.881$ (worst)	Strong systematic curve	Accelerating growth, opposite to real behaviour; 35-d value 47.4 MPa (+31%) is not credible

6.3 Interpretation

- **Best model: Logarithmic** ($f = 7.90 + 8.37 \ln t$). It has the highest adjusted R^2 (0.9930), the lowest RMSE (0.55 MPa) and by far the lowest cross-validation error (0.89 MPa vs 2.0 for quadratic), with just two parameters. It also matches the physics: rapid early hydration followed by a gradually decreasing rate of gain.
- **Quadratic** fits the tested range almost as well, but its fit is only valid within 3–28 d. Its downward turn after ~30 d is a mathematical artefact, so it should not be used for extrapolation.
- **Linear** and **exponential** are structurally wrong for this behaviour (both show curved residual patterns). Exponential growth is the worst choice because concrete strength gain decelerates.
- **Predictions.** At **t = 10 d** (interpolation, reliable): about **27.2 MPa** from the logarithmic model (24.0–27.2 MPa across all models; spread ≈ 3 MPa). At **t = 35 d** (extrapolation): about **37.7 MPa** from the logarithmic model, while the models range from 35.6 to 47.4 MPa. The 35-d value is therefore far less certain than the 10-d value.
- **Limitations.** Only 5 ages and 15 cubes; a few points drive the fit. The logarithmic form still grows without bound, so it should not be used far beyond ~35–60 d. Failure load to strength assumes a standard cube test and uniform loading. In practice, 35-d strength should be confirmed by testing, and design should use the 28-d characteristic strength.

6.4 Conclusion

Among the four regression models, the logarithmic model is the most accurate and the most physically reasonable. It predicts a compressive strength of about 27.2 MPa at 10 days and about 37.7 MPa at 35 days (a modest ~4% gain over 28 days).